

Final Project Report

Employee Lifecycle and Attrition Analytics Pipeline

Project Overview

The Employee Lifecycle and Attrition Analytics Pipeline is an end-to-end data engineering solution designed to process, track, and analyze employee records over time. By leveraging cloud data integration and a Medallion Architecture, this project transforms raw, fragmented HR data into clean, aggregated metrics that help an organization understand workforce dynamics and predict attrition risk.

Problem Statement

Modern HR teams face significant challenges in tracking the complete lifecycle of an employee. Data regarding promotions, department transfers, salary adjustments, and resignations is often fragmented across daily snapshots or isolated systems. Without a historical tracking mechanism, it is difficult to identify patterns in employee turnover, measure accurate retention rates, or flag departments with high attrition risk.

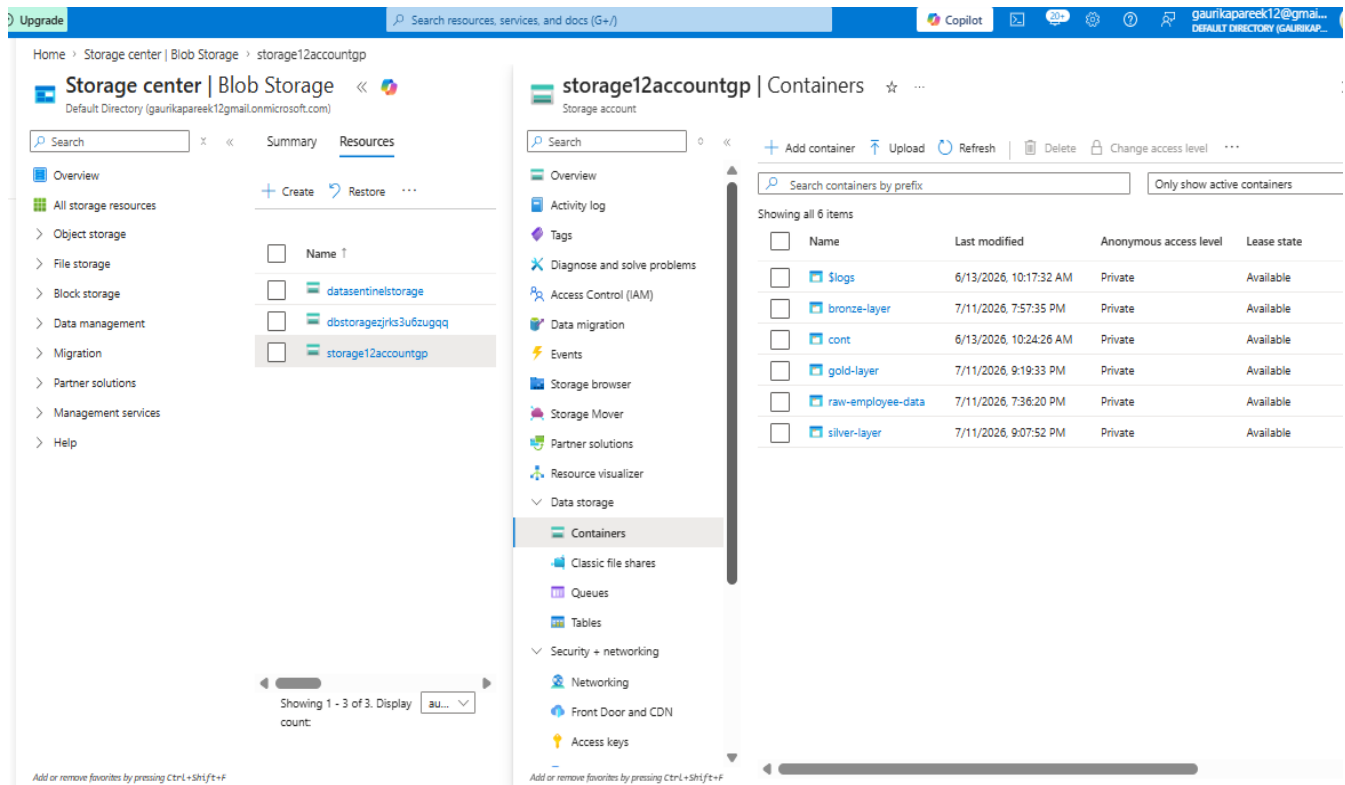
Proposed Solution

This project builds an automated data pipeline that processes daily batches of employee records. Instead of overwriting old data, the system tracks historical changes (such as a salary bump or a role change) using Slowly Changing Dimensions (SCD Type 2), while maintaining a current, up-to-date snapshot of the workforce. The final output provides aggregated analytics on headcount, average tenure, and daily/monthly attrition rates, giving HR leadership actionable insight.

Complete Technical Architecture & Tech Stack

The table below lists every technology used to build this project end to end.

Layer / Area	Technology	Purpose
Programming	SQL	Transformations — JOINS, GROUP BY, CASE, CTEs, Window Functions
Ingestion	Azure Storage Account Azure Data Factory (ADF)	Securely store raw daily files (Bronze layer) Automated pipelines moving data into cloud storage
Processing	PySpark	Read raw files, DataFrame transformations at scale
Workspace	Databricks	Cluster environment for writing/running all code
Storage Layer	Delta Lake	ACID-compliant updates, SCD2 MERGE support
Modeling	Medallion Architecture	Bronze (Raw) → Silver (Cleaned/Tracked) → Gold (Business-ready)
Modeling	SCD Type 2	Delta MERGE to keep a full historical log of employee changes



Output - Azure Portal → Storage Account → Containers list, showing bronze-layer, silver-layer, and gold-layer containers all present. (*proves the 3-container storage setup exists*)

Data Pipeline Implementation

Bronze Layer — Raw Data Ingestion

The notebook connects to the Azure Storage Account using a securely stored access key (see Section 7 — Security), then reads five days of raw daily employee CSV files (day 1 to day 5) from the bronze-layer container. Each day's file is tagged with a batch_day and batch_date column so downstream layers know exactly which snapshot every row came from.

```

11:23 PM (<1s) 1

#Connection Of Storage Account
storage_account_name = "storage12accountgp"
storage_account_key = "yQhF8dKEAkD52qyacCpz7/xYGnF8UR9VhxSSGgBsbE+YBj1Q4ixBUWozg4e1J5QWIOi4rmMprja3+ASTsm80uw=="

spark.conf.set(
    f"fs.azure.account.key.{storage_account_name}.blob.core.windows.net",
    storage_account_key
)

print("Storage connected successfully!")

Storage connected successfully!

```

Output- Storage Connected

```
11:23 PM (<1s) 2 Python
#Bronze Data (day-wise)
bronze_path = f"wasbs://{bronze_layer}@{storage_account_name}.blob.core.windows.net/"
files = dbutils.fs.ls(bronze_path)
for f in files:
    print(f.path)

wasbs://bronze-layer@storage12accountgp.blob.core.windows.net/day 1/
wasbs://bronze-layer@storage12accountgp.blob.core.windows.net/day 2/
wasbs://bronze-layer@storage12accountgp.blob.core.windows.net/day 3/
wasbs://bronze-layer@storage12accountgp.blob.core.windows.net/day 4/
wasbs://bronze-layer@storage12accountgp.blob.core.windows.net/day 5/
```

Output - Notebook cell output listing all 5 day-folders inside the bronze-layer container.

```
11:23 PM (10s) 3 Python
# Reading Day 1 Data
day1_path = f"wasbs://{bronze_layer}@{storage_account_name}.blob.core.windows.net/day 1/"

df_day1 = spark.read.csv(day1_path, header=True, inferSchema=True)

# Number of Rows and Schema
df_day1.printSchema()
df_day1.show(5)
print(f"Total rows: {df_day1.count()}")

(5) Spark Jobs
> df_day1: pyspark.sql.dataframe.DataFrame = [emp_id: integer, name: string ... 4 more fields]

root
 |-- emp_id: integer (nullable = true)
 |-- name: string (nullable = true)
 |-- department: string (nullable = true)
 |-- salary: integer (nullable = true)
 |-- join_date: date (nullable = true)
 |-- status: string (nullable = true)

+-----+-----+-----+-----+-----+
|emp_id|  name|department|salary| join_date|status|
+-----+-----+-----+-----+-----+
| 151|Emp_151| Marketing| 73963|2020-03-08|Active|
| 152|Emp_152| Marketing| 79253|2020-03-31|Active|
| 153|Emp_153|         IT| 83668|2021-10-16|Active|
| 154|Emp_154|  Finance| 84331|2020-06-28|Active|
| 155|Emp_155| Marketing| 90570|2021-12-28|Active|
+-----+-----+-----+-----+-----+

only showing top 5 rows

Total rows: 200
```

Output - Notebook cell output showing Day 1 schema (printSchema) + first 5 rows + total row count

Schema Drift Handling

Before combining all 5 days into a single DataFrame, each day's schema is compared against the Day 1 baseline. Any new or missing columns are printed as a warning instead of silently breaking the job. The union itself uses `unionByName(allowMissingColumns=True)` rather than a plain `union()`, so columns are matched by name (not position), and if a future day introduces or drops a column, the pipeline fills the gap with null instead of crashing.

```

> all_days_df: pyspark.sql.dataframe.DataFrame = [emp_id: integer, name: string ... 6 more fields]
> df: pyspark.sql.dataframe.DataFrame = [emp_id: integer, name: string ... 6 more fields]

day 2: schema matches baseline, no drift
day 3: schema matches baseline, no drift
day 4: schema matches baseline, no drift
day 5: schema matches baseline, no drift

+-----+-----+-----+-----+-----+-----+-----+
|emp_id|  name|department|salary| join_date|status|batch_day|batch_date|
+-----+-----+-----+-----+-----+-----+-----+
| 151|Emp_151| Marketing| 73963|2020-03-08|Active|   day 1|2026-07-07|
| 152|Emp_152| Marketing| 79253|2020-03-31|Active|   day 1|2026-07-07|
| 153|Emp_153|      IT| 83668|2021-10-16|Active|   day 1|2026-07-07|
| 154|Emp_154|  Finance| 84331|2020-06-28|Active|   day 1|2026-07-07|
| 155|Emp_155| Marketing| 90570|2021-12-28|Active|   day 1|2026-07-07|
| 156|Emp_156| Marketing| 47947|2020-07-23|Active|   day 1|2026-07-07|
| 157|Emp_157|      HR| 80039|2020-10-23|Active|   day 1|2026-07-07|
| 158|Emp_158| Marketing| 94302|2020-02-13|Active|   day 1|2026-07-07|
| 159|Emp_159|  Finance| 69834|2021-05-20|Active|   day 1|2026-07-07|
| 160|Emp_160| Marketing| 90745|2020-08-17|Active|   day 1|2026-07-07|
+-----+-----+-----+-----+-----+-----+-----+

only showing top 10 rows

Total rows across all days: 1050

```

Output -schema-drift check loop — should show ‘baseline schema set’ for Day 1 and ‘schema matches baseline, no drift’ (or any detected drift) for Days 2–5, followed by the combined row count (1050).

Silver Layer — SCD Type 2 History Tracking

Day 1 data initializes the Silver history Delta table with start_date, end_date ('9999-12-31' = still active), and an is_current flag. For Days 2–5, a MERGE operation compares the incoming day's data against the current record for each employee: if department, salary, or status changed, the old record is closed (end_date set, is_current = 0) and a new version is inserted with is_current = 1. This is the core SCD Type 2 mechanism that keeps full history without losing any past state.

```

> day1_df: pyspark.sql.dataframe.DataFrame = [emp_id: integer, name: string ... 6 more fields]
> silver_history_init: pyspark.sql.dataframe.DataFrame = [emp_id: integer, name: string ... 7 more fields]

Silver history table initialized with Day 1 data!

+-----+-----+-----+-----+-----+-----+-----+-----+
|emp_id|  name|department|salary| join_date|status|start_date| end_date|is_current|
+-----+-----+-----+-----+-----+-----+-----+-----+
| 151|Emp_151| Marketing| 73963|2020-03-08|Active|2026-07-07|9999-12-31|      1|
| 152|Emp_152| Marketing| 79253|2020-03-31|Active|2026-07-07|9999-12-31|      1|
| 153|Emp_153|      IT| 83668|2021-10-16|Active|2026-07-07|9999-12-31|      1|
| 154|Emp_154|  Finance| 84331|2020-06-28|Active|2026-07-07|9999-12-31|      1|
| 155|Emp_155| Marketing| 90570|2021-12-28|Active|2026-07-07|9999-12-31|      1|
+-----+-----+-----+-----+-----+-----+-----+-----+

only showing top 5 rows

```

Output - ‘Silver history table initialized with Day 1 data!’ showing the first 5 rows with start_date / end_date / is_current columns.

```

> changed_or_new: pyspark.sql.dataframe.DataFrame = [emp_id: integer, name: string ... 5 more fields]
> current_records: pyspark.sql.dataframe.DataFrame = [emp_id: integer, name: string ... 7 more fields]
> new_day_df: pyspark.sql.dataframe.DataFrame = [emp_id: integer, name: string ... 5 more fields]
> to_insert: pyspark.sql.dataframe.DataFrame = [emp_id: integer, name: string ... 7 more fields]

--- Processing day 2 ---
day 2: 205 incoming source rows
day 2: 83 new/changed records inserted (out of 205 incoming rows)

--- Processing day 3 ---
day 3: 210 incoming source rows
day 3: 103 new/changed records inserted (out of 210 incoming rows)

--- Processing day 4 ---
day 4: 215 incoming source rows
day 4: 101 new/changed records inserted (out of 215 incoming rows)

--- Processing day 5 ---
day 5: 220 incoming source rows
day 5: 107 new/changed records inserted (out of 220 incoming rows)

SCD Type 2 processing complete for all days!

```

Output - Day 2–Day 5 MERGE loop — should show, for each day, the incoming row count and the new/changed record count actually inserted.

```
> history_df: pyspark.sql.dataframe.DataFrame = [emp_id: integer, name: string ... 7 more fields]

Total history records: 594

+-----+-----+-----+-----+-----+-----+-----+-----+
|emp_id|name  |department|salary|join_date |status|start_date|end_date  |is_current|
+-----+-----+-----+-----+-----+-----+-----+-----+
|1      |Emp_1 |Sales     |46632 |2020-07-30|Active|2026-07-07|2026-07-08|0         |
|1      |Emp_1 |Marketing |46632 |2020-07-30|Active|2026-07-08|2026-07-09|0         |
|1      |Emp_1 |Finance   |46632 |2020-07-30|Active|2026-07-09|2026-07-11|0         |
|1      |Emp_1 |Finance   |54780 |2020-07-30|Active|2026-07-11|9999-12-31|1         |
+-----+-----+-----+-----+-----+-----+-----+-----+

Current active records: 220
```

Output - filtering employee_history for a single emp_id (e.g. emp_id = 1) ordered by start_date — shows that one employee's full change history across the 5 days.

Window Functions — Change Tracking

A dedicated SQL query uses LAG() to look up each employee's previous department and salary (partitioned by emp_id, ordered by start_date), plus ROW_NUMBER() to number each version. This surfaces exactly which employees changed department, and which received a raise, directly from the SCD2 history — fulfilling the Window Functions requirement from the tech stack.

```
Employees whose department changed at least once:
+-----+-----+-----+-----+-----+-----+-----+-----+
|emp_id| name|start_date|department|salary|status|prev_department|prev_salary|salary_change|version_number|
+-----+-----+-----+-----+-----+-----+-----+-----+
|1      |Emp_1|2026-07-08|Marketing|46632|Active|Sales          |46632      |0            |2            |
|1      |Emp_1|2026-07-09|Finance  |46632|Active|Marketing      |46632      |0            |3            |
|2      |Emp_2|2026-07-08|Finance  |65513|Active|HR             |65513      |0            |2            |
|2      |Emp_2|2026-07-10|Sales    |65513|Active|Finance        |65513      |0            |3            |
|3      |Emp_3|2026-07-10|IT       |93010|Active|Sales          |93010      |0            |3            |
|7      |Emp_7|2026-07-09|IT       |30599|Active|Sales          |30599      |0            |2            |
|7      |Emp_7|2026-07-11|HR       |30599|Active|IT             |30599      |0            |3            |
|10     |Emp_10|2026-07-08|HR       |96524|Active|Finance        |96524      |0            |2            |
|10     |Emp_10|2026-07-11|Sales    |102465|Active|HR             |102465     |0            |4            |
|11     |Emp_11|2026-07-09|Finance  |57277|Active|Sales          |57277      |0            |2            |
+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 10 rows

Employees who got a raise (salary_change > 0):
+-----+-----+-----+-----+-----+-----+-----+-----+
|emp_id| name|start_date|department|salary|status|prev_department|prev_salary|salary_change|version_number|
+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+-----+
```

Output - Notebook cell output of the LAG/ROW_NUMBER window-function query — both the 'department changed' table and the 'salary_change > 0' table.

Current Snapshot Table

Filtering the Silver history table to is_current = 1 produces a single current-state row per employee, with tenure_years computed as the difference between today's date and the original join_date.

```

> current_snapshot: pyspark.sql.dataframe.DataFrame = [emp_id: integer, name: string ... 8 more fields]
> current_snapshot_final: pyspark.sql.dataframe.DataFrame = [emp_id: integer, name: string ... 6 more fields]

Current snapshot created! Total rows: 220
+-----+-----+-----+-----+-----+-----+-----+
|emp_id| name|department|salary| join_date|status|current_version_start_date|tenure_years|
+-----+-----+-----+-----+-----+-----+-----+
| 19|Emp_19| IT| 36875|2022-08-05|Active| 2026-07-07| 3.93|
| 29|Emp_29| HR| 39637|2021-04-27|Active| 2026-07-07| 5.2|
| 31|Emp_31| Sales| 48997|2020-08-09|Active| 2026-07-07| 5.92|
| 33|Emp_33| Sales| 78953|2020-05-19|Active| 2026-07-07| 6.14|
| 36|Emp_36| Finance| 46673|2020-02-02|Active| 2026-07-07| 6.44|
+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows

```

Output - Current snapshot

Gold Layer — Business-Ready Aggregates

Three aggregate tables are produced from the Silver layer using SQL (CTEs, JOINS, CASE, GROUP BY):

- headcount_by_department — active headcount per department, per day
- attrition_summary — new resignations and attrition rate (%) per department, per day
- tenure_summary — average tenure (years) and employee count per department

```

> headcount_by_department: pyspark.sql.dataframe.DataFrame = [batch_day: string, date: date ... 2 more fields]
| day 2|2026-07-08| HR| 48|
| day 2|2026-07-08| IT| 35|
| day 2|2026-07-08| Marketing| 34|
| day 2|2026-07-08| Sales| 41|
| day 3|2026-07-09| Finance| 44|
| day 3|2026-07-09| HR| 48|
| day 3|2026-07-09| IT| 35|
| day 3|2026-07-09| Marketing| 36|
| day 3|2026-07-09| Sales| 38|
| day 4|2026-07-10| Finance| 41|
| day 4|2026-07-10| HR| 40|
| day 4|2026-07-10| IT| 41|
| day 4|2026-07-10| Marketing| 33|
| day 4|2026-07-10| Sales| 44|
| day 5|2026-07-11| Finance| 36|
| day 5|2026-07-11| HR| 45|
| day 5|2026-07-11| IT| 42|
| day 5|2026-07-11| Marketing| 26|
| day 5|2026-07-11| Sales| 47|
+-----+-----+-----+-----+

```

Output — headcount by department

```

> attrition_summary: pyspark.sql.dataframe.DataFrame = [batch_day: string, date: date ... 4 more fields]
+-----+-----+-----+-----+-----+-----+
|batch_day| date|department|resignations|headcount_at_risk|attrition_rate_pct|
+-----+-----+-----+-----+-----+-----+
| day 1|2026-07-07| Finance| 0| 50| 0.0|
| day 1|2026-07-07| HR| 0| 41| 0.0|
| day 1|2026-07-07| IT| 0| 38| 0.0|
| day 1|2026-07-07| Marketing| 0| 37| 0.0|
| day 1|2026-07-07| Sales| 0| 34| 0.0|
| day 2|2026-07-08| Finance| 0| 47| 0.0|
| day 2|2026-07-08| HR| 0| 48| 0.0|
| day 2|2026-07-08| IT| 0| 35| 0.0|
| day 2|2026-07-08| Marketing| 0| 34| 0.0|
| day 2|2026-07-08| Sales| 0| 41| 0.0|
| day 3|2026-07-09| Finance| 2| 46| 4.35|
| day 3|2026-07-09| HR| 3| 51| 5.88|
| day 3|2026-07-09| IT| 2| 37| 5.41|
| day 3|2026-07-09| Marketing| 1| 37| 2.7|
| day 3|2026-07-09| Sales| 1| 39| 2.56|
| day 4|2026-07-10| Finance| 3| 44| 6.82|
| day 4|2026-07-10| HR| 5| 45| 11.11|
| day 4|2026-07-10| IT| 1| 42| 2.38|

```

Output- attrition summary

```
(2) Spark Jobs

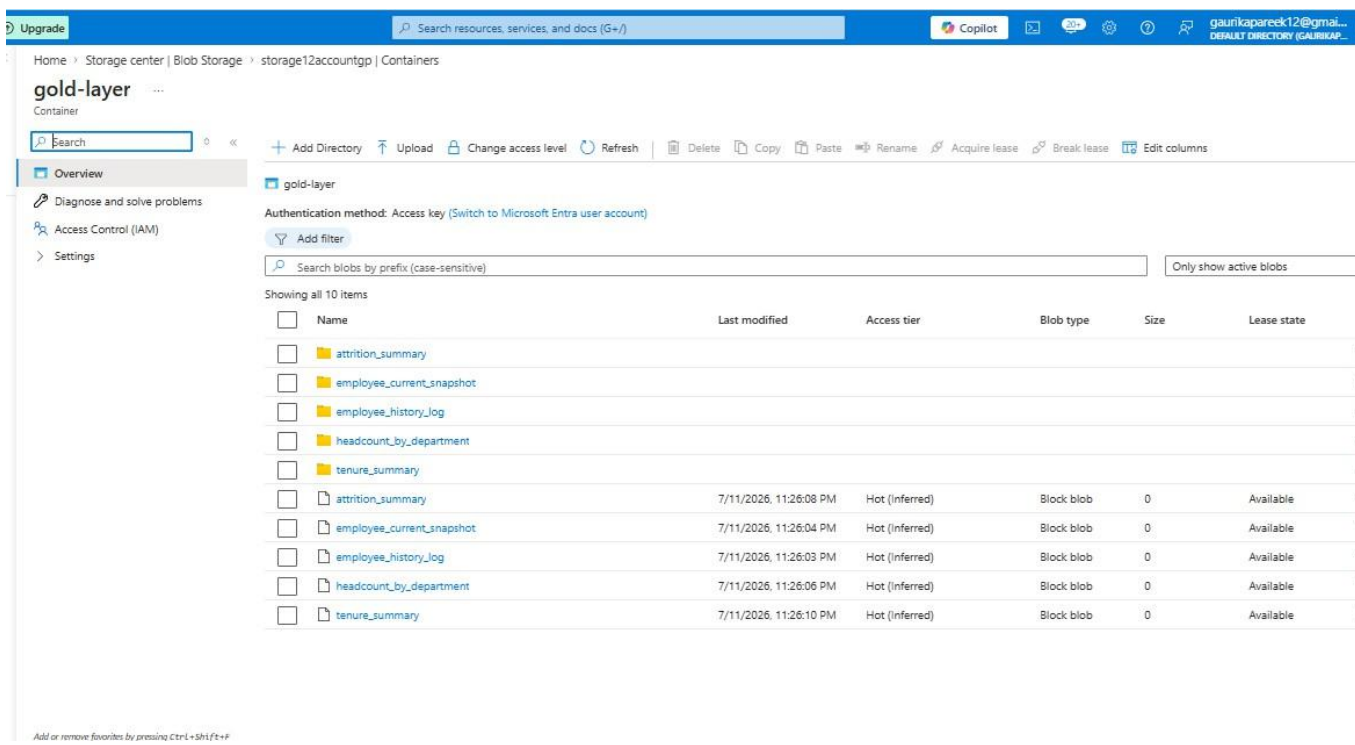
> tenure_summary: pyspark.sql.dataframe.DataFrame = [department: string, employee_count: long ... 1 more field]

+-----+-----+-----+
|department|employee_count|avg_tenure_years|
+-----+-----+-----+
| Finance |           36 |           4.56 |
|    HR   |           45 |           4.54 |
|    IT   |           42 |           4.4  |
| Marketing|           26 |           5.14 |
|   Sales |           47 |           5.14 |
+-----+-----+-----+
```

Output – tenure summary

Writing Outputs to the Gold Container

All five final tables (employee_history_log, employee_current_snapshot, headcount_by_department, attrition_summary, tenure_summary) are written out as CSV to the gold-layer container so they can be picked up by a BI/dashboard tool.



Output- Azure Storage Explorer view of the gold-layer container, showing all 5 output folders/files after the notebook has finished running.

Data Validation / Sanity Checks

A final sanity-check cell re-reads the Silver and Gold tables and prints: total history records, current snapshot record count, active vs resigned counts, the tenure summary (to cross-check against the saved CSV), and a distinct department list — confirming no department was dropped or duplicated during processing.

```
=== SANITY CHECK SUMMARY ===
1. Total history records: 594 (Reference: 594)
2. Current snapshot records: 220 (Reference: 220)
3. Total active employees (day 5): 196
4. Total resigned employees (day 5): 24

=== Tenure Summary (compare with tenure_summary.csv) ===
+-----+-----+-----+
|department|employee_count|avg_tenure_years|
+-----+-----+-----+
|  Finance |           36 |           4.56 |
|    HR    |           45 |           4.54 |
|    IT    |           42 |           4.4  |
|Marketing |           26 |           5.14 |
|   Sales  |           47 |           5.14 |
+-----+-----+-----+

=== Department count check ===
+-----+
|department|
+-----+
|  Finance |
|    HR    |
|    IT    |
|Marketing |
|   Sales  |
+-----+
```

Output- === SANITY CHECK SUMMARY ===

Conclusion

The pipeline successfully demonstrates a complete Bronze → Silver → Gold Medallion Architecture on Azure Databricks, using Delta Lake MERGE operations to implement true SCD Type 2 history tracking, and SQL (CTEs, JOINS, CASE, GROUP BY, window functions) to produce business-ready attrition and tenure analytics. With the fixes above applied — secure credential handling, corrected attrition math, and schema-drift resilience — the pipeline is reliable, reproducible, and ready to support HR decision-making on workforce trends and retention risk.

Final Output Row Counts

Output Table	Rows	Notes
employee_history_log	594	Full SCD2 history, all versions of every employee
employee_current_snapshot	220	One current row per employee (matches total unique emp_ids by Day 5)
headcount_by_department	25	5 days × 5 departments
attrition_summary	25	5 days × 5 departments
tenure_summary	5	One row per department